

Application identity: Cognito, presigned URLs and Verified Permissions

Identity for people who are not your staff – customers, partners and the applications they use.

A user pool answers **who are you**. An identity pool answers **what AWS credentials do you get**. Most apps need only the first.

User pool, or identity pool?

USER POOL

Authentication

- A managed user directory
- Sign-up, sign-in, MFA, federation
- Issues ID, access and refresh tokens
- Your API validates the token

IDENTITY POOL

Authorization to AWS

- Exchanges a token for AWS credentials
- Authenticated and unauthenticated roles
- Client calls S3 or DynamoDB directly
- Only needed when the client calls AWS

THE TIEBREAKER

'The mobile client uploads directly to S3' is the phrase that adds an identity pool. Without it, a user pool alone is enough.

The three tokens

- ◆ **ID token** – who the user is. Claims like email, name, and **cognito:groups**. For your application to identify the user.
- ◆ **Access token** – what they may do. OAuth scopes. This is what an API should authorise against.
- ◆ **Refresh token** – obtains new ID and access tokens without re-authenticating. Long-lived, so treat it as a credential.
- ◆ **Cognito groups** add a **cognito:groups** claim, which is coarse-grained authorisation. Fine-grained needs Verified Permissions or your own logic.
- ◆ Keep ID and access token validity short – an hour is a sensible ceiling. A 24-hour access token is a 24-hour window.

ON THE EXAM

Refresh token rotation arrived April 2025, limiting damage from a stolen refresh token.

What changed in November 2024

- ◆ **Managed Login** replaced the classic hosted UI, with a no-code branding editor.
- ◆ **Passwordless authentication** arrived natively – **passkeys, email OTP and SMS OTP**.
- ◆ **Feature tiers: Lite, Essentials and Plus**. Capability now depends on which tier you are on.
- ◆ **Advanced security features were renamed threat protection** and moved into the **Plus** tier.
- ◆ Since then: refresh token rotation (April 2025), WAF for Managed Login (June 2025), **PrivateLink (November 2025)**, and multi-Region replication with customer managed keys (May 2026).

ON THE EXAM

Dated: if your material says 'advanced security features', it predates November 2024. The capability is the same; the name and the tier are not.

Threat protection, and what it does not cover

- ◆ **Compromised credentials** – compares sign-ins against public breach data and guessable passwords.
- ◆ **Adaptive authentication** scores risk per sign-in. Plus **IP allow and deny lists**.
- ◆ **Audit mode publishes CloudWatch metrics and mitigates nothing.** Enforced mode acts.
- ◆ **Compromised credentials are not detected in SRP or custom auth** – username-and-password only.
- ◆ **MFA and threat protection apply to local users.** Federated users are the IdP's responsibility.

ON THE EXAM

Mirrors module 22: whoever authenticates the user protects the authentication.

Identity pool roles

- ◆ An **authenticated role** for signed-in users, and an optional **unauthenticated role** for guests.
- ◆ The **unauthenticated role is a real attack surface**. Anyone can obtain those credentials.
- ◆ **Role-based access control** picks a role from token claims.
- ◆ **Attributes for access control** maps claims to **principal tags** – one role, per-user scope.
- ◆ Never s3:* on an authenticated role. Use policy variables for per-user prefixes.

ON THE EXAM

One role plus a principal tag is how a million users share a policy.

Presigned URLs

- ◆ One object, one operation, signed by a principal. The holder acts **as that principal**.
- ◆ **It inherits the signer's permissions.** If the signer cannot read it, neither can the URL.
- ◆ **It expires at the earlier of your expiry or the signing credentials' expiry.**
- ◆ Max 7 days with an IAM user's long-term credentials. Far shorter with a role.
- ◆ **There is no revocation.** Short expiry is the only control you keep.

ON THE EXAM

A presigned URL is a bearer token in a link.

Amazon Verified Permissions

- ◆ Managed **authorization** for decisions **inside your application** – may this user edit this document.
- ◆ Policies in **Cedar**, the same language as AWS Verified Access in module 15.
- ◆ A **Cognito user pool can be an identity source**. AVP validates the token and evaluates the claims.
- ◆ Policy is separate from code, so an entitlement change is not a release.
- ◆ It says nothing about who can reach the app, or about AWS API calls.

ON THE EXAM

IAM authorises AWS API calls. Verified Permissions authorises your application's actions.

A photo-sharing mobile app needs users to register and sign in with email or Google. Once signed in, the app must upload large images directly from the device to an S3 bucket, without routing the bytes through the company's API servers.

Each user must be able to write only to their own prefix in the bucket, and there are around two million users.

What should the company implement?

- ☐ A A Cognito user pool for sign-in, and an identity pool issuing credentials scoped by a policy variable to each user's prefix
- ☐ B A Cognito user pool only; the app validates the token and calls S3 with the token
- ☐ C An IAM user per app user, with a policy scoped to that user's prefix
- ☐ D A Cognito identity pool only, with the unauthenticated role granting s3:PutObject

◆ PAUSE AND DECIDE

What should the company implement?

- A

 User pool for sign-in, identity pool for scoped AWS credentials
Correct. The user pool authenticates; the identity pool converts that into AWS credentials; a policy variable gives two million users one policy and their own prefix.
- B

 User pool only, calling S3 with the token
S3 does not accept a Cognito token. AWS APIs want SigV4-signed requests with AWS credentials – the same gap the IoT credential provider closed in module 13.
- C

 An IAM user per app user
Two million IAM users, far past the account quota, each with long-lived credentials. This is the anti-pattern module 22 opened by arguing against.
- D

 Identity pool with an unauthenticated role
The unauthenticated role is available to anyone at all. Granting PutObject on it means the internet can write to your bucket.

THE SIGNAL

'Directly from the device' is the phrase that adds the identity pool. Without it, a user pool alone would be enough.

A media company issues presigned S3 URLs for premium video files. A Lambda function, using its execution role, generates each URL with an expiry of 12 hours so subscribers can download over a slow connection.

Subscribers report that download links stop working after about an hour, well before the stated expiry. The bucket policy, the object and the role's permissions are all unchanged during that period.

What is the cause?

- ☐ A S3 caps presigned URL validity at 1 hour for all signing methods
- ☐ B The bucket has an S3 Lifecycle rule transitioning objects to Glacier
- ☐ C The Lambda function's concurrency limit is invalidating in-flight URLs
- ☐ D The URL expires with the role session's temporary credentials, whichever comes first

◆ PAUSE AND DECIDE

What is the cause?

- A

 S3 caps all presigned URLs at 1 hour
No such universal cap. Signed with an IAM user's long-term credentials, a presigned URL can last up to 7 days - the ceiling comes from the credentials, not from S3.
- B

 A Lifecycle rule to Glacier
Would break every download of an object once transitioned, not links at a consistent age. Transitions also run on a schedule, not per-link.
- C

 Lambda concurrency invalidating URLs
Concurrency throttles invocations. Once a URL is generated it is a signed string with no relationship to the function that made it.
- D

 The URL dies with the role session's credentials
Correct. A presigned URL is valid for the shorter of your expiry and the signing credentials' lifetime. A Lambda execution role session is around an hour.

THE SIGNAL

'About an hour' against a requested twelve is the tell. That is a credential lifetime, not a configured expiry.

A legal document platform authenticates users with a Cognito user pool. Authorisation is currently hardcoded: each service checks whether the user is in the reviewer or the owner Cognito group.

The business now needs rules like 'a reviewer may comment on a document only while it is in review, and only if they are not its author'. Rules change monthly, and each change currently requires a code release across four services.

What should the platform adopt?

- ☐ A More Cognito groups, one per combination of role and document state
- ☐ B An identity pool with role-based access control selecting a role per group
- ☐ C Amazon Verified Permissions, with the user pool as an identity source and rules in Cedar
- ☐ D IAM policies with ABAC conditions on `aws:PrincipalTag`

◆ PAUSE AND DECIDE

What should the platform adopt?

- A

 More Cognito groups per combination
Groups are coarse and static. Document state changes constantly, and 'not the author' is a relationship between user and record that no group can express.
- B

 Identity pool with role-based access control
Selects an AWS role from a claim. The decision here is about a document in the application, which no IAM role evaluates.
- C

 Verified Permissions with Cedar and the user pool as identity source
Correct. Per-request decisions using user claims, resource attributes and context – and policy lives outside the code, so a rule change is not a release.
- D

 IAM policies with aws:PrincipalTag
ABAC governs AWS API calls. Commenting on a document is an application action against an application object – IAM never sees it.

THE SIGNAL

'Only while it is in review' and 'only if they are not its author' are resource state and relationship. IAM evaluates neither.

A retailer's Cognito user pool is on the Plus feature plan with threat protection enabled in enforced mode. Around 70% of its users are local accounts; the remaining 30% federate from a partner's SAML identity provider.

After a credential-stuffing campaign, the security team reports that compromised-credential detection and adaptive authentication fired for local users but produced nothing at all for the federated ones.

Why?

- | | |
|--|---|
| <p>A Threat protection must be enabled separately on each app client for federated users</p> | <p>B Threat protection applies to local users; the third-party IdP owns the security posture of federated users</p> |
| <p>C Federated users need the Essentials tier, which covers external identity providers</p> | <p>D Adaptive authentication runs only in audit mode for SAML sign-ins</p> |

◆ PAUSE AND DECIDE

Why?

- A

 Enable it per app client for federated users
App-client-level threat protection settings do exist, which makes this plausible. They change the configuration for local users – they do not extend it to federated ones.
- B

 It applies to local users; the IdP owns federated users
Correct. AWS states this directly. Cognito never sees the federated user's password, so it cannot compare it against breach data or score the sign-in.
- C

 Federated users need the Essentials tier
Backwards – Plus is the higher tier and already includes threat protection. No tier extends it to identities Cognito does not authenticate.
- D

 Adaptive auth is audit-only for SAML
Invented. The mode is a setting on the user pool, not something that silently changes by sign-in type.

THE SIGNAL

The same boundary as module 22's Identity Center nuance: whoever authenticates the user is who can protect the authentication.

Five things to carry forward

- ◆ **User pool authenticates. Identity pool issues AWS credentials.** Only add the second when the client calls AWS directly.
- ◆ **A presigned URL expires with the signing credentials,** whichever comes first, and cannot be revoked.
- ◆ **Threat protection covers local users only.** Federated users are the IdP's responsibility.
- ◆ **Audit mode mitigates nothing.** It publishes metrics and lets everything through.
- ◆ **Verified Permissions authorises inside your application.** IAM authorises AWS API calls. Different layers.

ON THE EXAM

Next: ABAC, permissions boundaries and least privilege at scale.